ISO/IEC JTC1 SC22 WG21 P00211r0
Date: 2016-01-30
To: LEWG

Allocator-aware library wrappers for dynamic allocation

Thomas Köppe <tkoeppe@google.com>

Contents

Summary

Short form: We propose to add library functions that allow the systematic use of allocators as a customisation point for dynamic allocations. The new functions complete the following picture:

	using `operator {new,delete}`	using allocator
Manual	`T * p = new T(args...)`	`auto p = allocate_new<T>(alloc, args...)` new!
	`delete p`	`allocate_delete(alloc, p)` new!
Unique pointer	`make_unique<T>(args...)`	`allocate_unique<T>(alloc, args...)` new!
	`default_delete<T>`	`allocator_delete<T>` new!
Shared pointer	`make_shared<T>(args...)`	`allocate_shared<T>(alloc, args...)`

Long form: The standard library rarely uses `new`/`delete` directly, but instead allows customisation of dynamic allocation and object construction via allocators. Currently this customisation is only available for container *elements* and for `shared_ptr` (as well as for a few other types that require dynamically allocated memory), but not for the top-level objects themselves.

The proposal is to complete the library facilities for allocator-based customisation by providing a direct mechanism for creating and destroying a dynamically stored object through an allocator, as well as a new deleter type for `unique_ptr` to hold an allocator-managed unique pointee, together with the appropriate factory function.

Example use cases

- Consider an arena allocator. A vector may put its elements into the arena, but the vector itself cannot easily be placed in the same arena. The proposed `std::allocate_unique<std::vector<T, ScopedArenaAlloc<T>>>(arena_alloc)` allows this. (Here we assume the use of the usual alias idiom `template <typename T> using ScopedArenaAlloc = std::scoped_allocator_adaptor<ArenaAlloc<T>>`;).
- Consider a shared memory allocator. As in the previous example, containers will put their elements in shared memory, but one process needs to create the container itself and place it in shared memory. This is now possible with `auto p = std::allocate_new<std::vector<T, ScopedShmemAlloc<T>>>(shmem_alloc)`. The returned pointer is presumably an offset pointer, and its offset value needs to be communicated to the other participating processes. The last process to use the container calls `allocate_delete(shmem_alloc, p)`.
- Allocator support is frequently requested on Stack Overflow.

Dynamic allocation wrappers

Allocation and object creation

```
template <typename A, typename ...Args>
auto allocate_new(A& alloc, Args&&... args) {
  using Traits = allocator_traits<A>;

  auto p = Traits::allocate(alloc, 1);
  if (!p) throw bad_alloc();

  try {
    Traits::construct(alloc, addressof(*p), std::forward<Args>(args)...);
  } catch(...) {
    Traits::deallocate(alloc, p, 1);
    throw;
  }

  return p;
}
```

Object destruction and Deallocation

```
template <typename A>
void allocate_delete(A& alloc, typename allocator_traits<A>::pointer p) {
  using Traits = allocator_traits<A>;

  Traits::destroy(alloc, addressof(*p));
  Traits::deallocate(alloc, p, 1);
}
```

Unique pointers

To allow `std::unique_ptr` to use custom allocators, we first need a deleter template that stores the allocator:

```
template <typename A>
struct allocator_delete {
  using pointer = typename allocator_traits<A>::pointer;

  A a_;  // exposition only

  allocator_delete(const A& a) : a_(a) {}

  void operator()(pointer p) {
    allocate_delete(a_, p);
  }
};
```

The factory function is:

```
template <typename T, typename A, typename ...Args>
unique_ptr<T, allocator_delete<A>> allocate_unique(A& alloc, Args&&... args) {
  return unique_ptr<T, allocator_delete<A>>(allocate_new<T>(alloc, std::forward<Args>
(args)...), alloc);
}
```

The type `T` must not be an array type.

Design questions

- Require exact allocator or allow rebind-copying? Rebind-copying would allow taking the original allocator by const-reference, but would incur a local allocator's destruction (which is not required to be `noexcept`).
- Related: Should `allocate_new` take an explicit typename `T` as a parameter, or use `A::value_type`?
- We could have reused the name `allocate_delete` for both for the function template that replaces `delete` and for the class template that replaces `default_delete` to keep using the word "allocate" consistently. That would require saying the unwieldy `struct allocate_delete` in the `allocate_unique` function template, though.